Comprehensive Overview: 520 Cox Pipeline

This document is the single source of truth for how `520_pipeline_cox_working.ipynb` runs end to end, which files it reads/writes, and how configuration and overrides control the workflow. It restores the detailed algorithmic explanations from earlier versions, updated to reflect the current, mode specific (fwd/bwd) pipeline.

Executive Summary

The pipeline transforms raw snapshot data into a time varying survival dataset for Cox regression, integrating real OER data, explicit forward/backward performance metrics, pool based comparisons, and configurable modeling/plotting. The workflow is modular, checkpointed, and designed for repeatable runs with run specific overrides.

Pipeline Architecture

High Level Flow

```
Raw Snapshots (502 output or database/file)
  -> Phase 1: Load and Base Prep
  -> Phase 2: OER Assignment and Pool Metrics (fwd/bwd only)
  -> Phase 3: Variable Creation
  -> Phase 4: Filtering
  -> Phase 5: Cox Analysis, Plots, Models
```

High Level Flow (Detailed, Recreated)

```
Raw Snapshot Data (502 output OR database/file)
    ↓
[PHASE 1] Data Loading & Basic Preparation
    ├─ CELL 1: Load raw snapshot data → df_pipeline_01_raw
    ├─ CELL 2: Basic filtering → df_pipeline_02_base
    └─ CELL 3: Calculate year group (yg) → df_pipeline_03_base
    ↓
[PHASE 2] OER Assignment & Pool Metrics (fwd/bwd only)
    ├─ CELL 4: Assign OERs (fwd/bwd) → df_pipeline_04a_basic_metrics
    ├─ CELL 5: Pool means/minus means/sizes → df_pipeline_05_pool_means
    └─ CELL 6: Pool ranks/pct/z scores → df_pipeline_06_pool_ranks
    ↓
[PHASE 3] Variable Creation
    ├─ CELL 7: Create time varying variables → df_pipeline_07_time_varying
    └─ CELL 8: Create static variables → df_pipeline_08_combined
    ↓
[PHASE 4] Advanced Filtering
    └─ CELL 9: Advanced filtering → df_pipeline_09_filtered
    ↓
[PHASE 5] Cox Analysis Preparation & Execution
    ├─ CELL 10: Cox data preparation → df_pipeline_10_cox_ready
    ├─ CELL 10.5: Standardize + interactions → df_pipeline_10_5_cox_zscored
    ├─ CELL 11: Cox analysis & plotting → df_pipeline_11_cox_analysis + plots
    └─ CELL 12: Cox regression models (scikit-survival)
        ├─ 12.1: Prepare data → df_pipeline_12_01_prepared
        ├─ 12.2: Fit static model
        ├─ 12.3: Fit full model
        ├─ 12.4: Model comparison & signal ratios
        ├─ 12.5: Competing risks analysis
        ├─ 12.6: Partial effects plots
        ├─ 12.6A: Combined/diagnostic effect plots
        └─ 12.7: Interaction diagnostics
```

Module Interaction Diagram (Phase 2)

```
┌──────────────────────────────────────────────────────────────────────────┐
|                    PHASE 2: OER INTEGRATION FLOW                         |
└──────────────────────────────────────────────────────────────────────────┘

[CELL 4] Assign OERs (fwd/bwd) -> df_pipeline_04a_basic_metrics
    |
    ├─ Input: df_pipeline_03_base + df_oer_enriched
    ├─ Algorithm: temporal lookups for forward/backward assignment
    |
    ↓
[CELL 5] Pool Means/Minus Means/Sizes -> df_pipeline_05_pool_means
    |
    ├─ Input: df_pipeline_04a_basic_metrics
    ├─ Algorithm: groupby by (snpsht_dt, pool_id)
    |
    ↓
[CELL 6] Pool Ranks/Pct/Z Scores -> df_pipeline_06_pool_ranks
    |
    ├─ Input: df_pipeline_05_pool_means
    ├─ Algorithm: rank + pct + z within pool
```

Phase 2 Detailed Flow (Verbatim Style, Updated)

```
[CELL 4] Assign OERs (fwd/bwd) -> df_pipeline_04a_basic_metrics
    |
    ├─ Input: df_pipeline_03_base + df_oer_enriched
    |
    ├─ Algorithm:
    |   • Normalize OER dates (eval_strt_dt, eval_thru_dt)
    |   • Deduplicate OER rows per officer/eval window
    |   • Forward assignment: last eval_thru_dt <= snpsht_dt
    |   • Backward assignment: next eval_thru_dt >= snpsht_dt
    |   • Compute cum_evals, cum_tb, tb_ratio for rtr/snr (fwd + bwd)
    |   • Important rule: if no box check, do not increment cum_evals
    |
    └─ Output: df_pipeline_04a_basic_metrics.feather
        |
        ↓
[CELL 5] Pool Means/Minus Means/Sizes -> df_pipeline_05_pool_means
    |
    ├─ Input: df_pipeline_04a_basic_metrics
    |
    ├─ Algorithm:
    |   • Group by (snpsht_dt, rtr_rater) and (snpsht_dt, snr_rater)
    |   • Compute pool mean TB ratio (fwd + bwd)
    |   • Compute pool size
    |   • Compute minus mean (individual - pool mean)
    |
    └─ Output: df_pipeline_05_pool_means.feather
        |
        ↓
[CELL 6] Pool Ranks/Pct/Z Scores -> df_pipeline_06_pool_ranks
    |
    ├─ Input: df_pipeline_05_pool_means
    |
    ├─ Algorithm:
    |   • Group by the same pool keys as Cell 5
    |   • Rank ascending (higher percentile = better rank)
    |   • Compute percentile rank and pool z score
    |
    └─ Output: df_pipeline_06_pool_ranks.feather
```

Phase 1 Detailed Flow (Verbatim Style, Updated)

```
[CELL 1] Load Raw Snapshots -> df_pipeline_01_raw
    |
    ├─ Input: 502 output OR database/file
    ├─ Algorithm:
    |   • Load snapshot data into a DataFrame
    |   • Normalize basic dtypes and date fields
    |
    ├─ Time Complexity: O(S) where S = snapshot rows
    └─ Output: df_pipeline_01_raw.feather
        |
        ↓
[CELL 2] Base Filtering -> df_pipeline_02_base
    |
    ├─ Input: df_pipeline_01_raw
    ├─ Algorithm:
    |   • Filter to commissioned officers
    |   • Remove problematic job codes (ever observed)
    |
    ├─ Time Complexity: O(S)
    └─ Output: df_pipeline_02_base.feather
        |
        ↓
[CELL 3] Year Group (yg) -> df_pipeline_03_base
    |
    ├─ Input: df_pipeline_02_base
    ├─ Algorithm:
    |   • Compute fiscal year from appointment date
    |   • Exclude inconsistent multi-yg officers
    |   • Map yg to all snapshots
    |
    ├─ Time Complexity: O(S)
    └─ Output: df_pipeline_03_base.feather
```

Phase 3 Detailed Flow (Verbatim Style, Updated)

```
[CELL 7] Time Varying Variables -> df_pipeline_07_time_varying
    |
    ├─ Input: df_pipeline_06_pool_ranks
    ├─ Algorithm:
    |   • Create time varying covariates (unit, division, job code, etc.)
    |   • Map time varying classifications to each snapshot
    |
    ├─ Time Complexity: O(S)
    └─ Output: df_pipeline_07_time_varying.feather
        |
        ↓
[CELL 8] Static Variables -> df_pipeline_08_combined
    |
    ├─ Input: df_pipeline_07_time_varying
    ├─ Algorithm:
    |   • Derive officer-level covariates (sex, age at rank, final job code)
    |   • Merge static values back to all snapshots
    |
    ├─ Time Complexity: O(S)
    └─ Output: df_pipeline_08_combined.feather
```

Phase 4 Detailed Flow (Verbatim Style, Updated)

```
[CELL 9] Advanced Filtering -> df_pipeline_09_filtered
    |
    ├─ Input: df_pipeline_08_combined
    ├─ Algorithm:
    |   • Apply filtering_params (dates, job codes, divisions, yg, gender)
    |   • Report counts and verify required columns
    |
    ├─ Time Complexity: O(S)
    └─ Output: df_pipeline_09_filtered.feather
```

Phase 5 Detailed Flow (Verbatim Style, Updated)

```
[CELL 10] Survival Interval Creation -> df_pipeline_10_cox_ready
    |
    ├─ Input: df_pipeline_09_filtered
    ├─ Algorithm:
    |   • Sort snapshots per officer
    |   • Build [start_time, stop_time] intervals
    |   • Assign event (promotion, attrition, censored)
    |   • Attach time-varying covariates to each interval
    |
    ├─ Time Complexity: O(O × I) where O = officers, I = intervals per officer
    └─ Output: df_pipeline_10_cox_ready.feather
        |
        ↓
[CELL 10.5] Standardize + Interactions -> df_pipeline_10_5_cox_zscored
    |
    ├─ Input: df_pipeline_10_cox_ready
    ├─ Algorithm:
    |   • Z-score configured columns (if enabled)
    |   • Build interaction and quadratic terms (if configured)
    |
    ├─ Time Complexity: O(S)
    └─ Output: df_pipeline_10_5_cox_zscored.feather
        |
        ↓
[CELL 11] Cox Plots & Analysis Dataset -> df_pipeline_11_cox_analysis
    |
    ├─ Input: df_pipeline_10_5_cox_zscored
    ├─ Algorithm:
    |   • Select columns via COLUMN_CONFIG
    |   • Create dummy variables (if enabled)
    |   • Generate CR/KM plots and CIF bar plots
    |
    ├─ Time Complexity: O(G × T) where G = groups, T = time points
    └─ Output: df_pipeline_11_cox_analysis.feather + plots
        |
        ↓
[CELL 12] Cox Regression Models
    |
    ├─ 12.1 Prepare Data
    |   • Validate structure and events
    |   • Remove invalid intervals
    |   • Time Complexity: O(S)
    |
    ├─ 12.2 Static Model
    |   • Officer-level model
    |   • Time Complexity: O(N × V^2)
    |
    ├─ 12.3 Full Model (static + time varying)
    |   • Interval-level model
    |   • Time Complexity: O(N × V^2)
    |
    ├─ 12.4 Model Comparison
    |   • Signal ratios, plots, comparisons
    |   • Time Complexity: O(V)
    |
    ├─ 12.5 Competing Risks
    |   • Separate promotion/attrition models
    |   • Time Complexity: O(2 × N × V^2)
    |
    ├─ 12.6 Partial Effects
    |   • Curves and bar plots for key variables
    |   • Time Complexity: O(K × T)
    |
    ├─ 12.6A Combined/Diagnostic Effects
    |   • Linear + quadratic combined plots
    |   • Time Complexity: O(K × T)
    |
```

```
    └─ 12.7 Interaction Diagnostics
        • Optional interaction analysis
        • Time Complexity: O(K × T)
```

Key Design Principles

1. Modularity: Each cell is independently runnable using flags.
2. Progressive saves: Each phase saves intermediate results for restartability.
3. Time integrity: OER assignment honors temporal rules for forward and backward metrics.
4. Pool correctness: Pool metrics are computed from explicit pools, not inferred.
5. Configuration driven: Plotting and modeling are controlled by config dictionaries.

Pipeline Phases and Outputs

Phase 1: Data Loading and Base Prep

- Cell 1: Load raw snapshots -> `df_pipeline_01_raw`
- Cell 2: Base filtering -> `df_pipeline_02_base`
- Cell 3: Add year group (yg) -> `df_pipeline_03_base`

Phase 2: OER Assignment and Pool Metrics

- Cell 4: Assign OERs (fwd/bwd) -> `df_pipeline_04a_basic_metrics`
  - Function: `assign_oer_to_snapshots_fast()` in `add_cum_oer_metrics_mod_working.py`
- Cell 5: Pool means/minus means/sizes -> `df_pipeline_05_pool_means`
  - Function: `add_pool_means_and_sizes()`
- Cell 6: Pool ranks/pct/z scores -> `df_pipeline_06_pool_ranks`
  - Function: `add_pool_ranks_pct_zscores()`

Phase 3: Variable Creation

- Cell 7: Time varying variables -> `df_pipeline_07_time_varying`
- Cell 8: Static variables -> `df_pipeline_08_combined`

Phase 4: Filtering

- Cell 9: Advanced filtering -> `df_pipeline_09_filtered`

Phase 5: Cox Analysis

- Cell 10: Survival interval creation -> `df_pipeline_10_cox_ready`
- Cell 10.5: Standardize + interactions -> `df_pipeline_10_5_cox_zscored`
- Cell 11: Cox plots and analysis -> `df_pipeline_11_cox_analysis`
- Cell 12: Cox models -> `df_pipeline_12_01_prepared`
  - 12.1: Prepare data for Cox

- 12.2: Static model
- 12.3: Full model
- 12.4: Model comparison
- 12.5: Competing risks analysis
- 12.6: Partial effects
- 12.6A: Combined/diagnostic effect plots
- 12.7: Interaction diagnostics

Configuration System

Base Config

- `pipeline_config.py` is always loaded first.
- Contains: execution flags, column definitions, filtering params, plot specs, constants.

Run Overrides

- `pipeline_config_16_1.py`, `pipeline_config_16_2.py`, `pipeline_config_16_3.py` (and optional others).
- Loaded on top of the base config via the notebook loader (`pip_config_file` in Cell 0).
- Overrides use **RUN_SCALE** (`'raw'` or `'z'`), **RUN_PROFILE** (tv_vars, quadratic_bases, interaction_name, plot_prefix, cr_plot_defaults), and **expand_run_profile(RUN_SCALE, RUN_PROFILE)** so one place controls model vars, standardization, quadratics, interaction config, and automated CR plots. **RUN_CR_PLOT_DEFAULTS** sets bins etc. for all CR plots in one swipe; **USE_OVERRIDE_CR_PLOTS** (default False) lets the override supply an explicit CR plot list for per-plot editing. See `current_documents/PIPELINE_CONFIG_OVERVIEW.md` for full run profile details.
- Run-specific: filtering params, plot palettes/flags, and (for 16_1) optional legacy OER column in time_varying_cols.

Algorithmic Flow and Key Patterns

Forward/Backward Options (Individual and Pool)

Individual metrics are computed in Cell 4 for **both directions**: - **Forward (fwd)**: last completed OER as of `snpsht_dt` (known at the time). - **Backward (bwd)**: next OER after `snpsht_dt` (painted back across its rating period).

This yields parallel columns for rater and senior rater: `cum_evals_fwd_*`, `cum_tb_fwd_*`, `tb_ratio_fwd_*` and their `*_bwd_*` counterparts.

Pool metrics in Cells 5–6 **inherit the direction from the individual ratios**: - Pool means/minus means/sizes are computed **for both fwd and bwd** TB ratios. - Pool ranks/pct/z scores are computed **for both fwd and bwd** TB ratios.

This is what enables mixing: you can model an **individual forward** metric with a **pool backward** metric in the same Cox run by selecting the columns in `pipeline_config.py` or `pipeline_config_15_x.py`.

Forward/Backward Decision Matrix

Use Case	Individual Metric	Pool Metric	Interpretation
Predictive signal of future evaluation	Forward	Forward	Both individual and pool reflect what was known as of the snapshot date.
Performance period attribution	Backward	Backward	Both individual and pool reflect the eventual evaluation for the period.
Individual forward vs pool backward	Forward	Backward	Individual is predictive; pool reflects eventual period outcomes.
Individual backward vs pool forward	Backward	Forward	Individual reflects period outcome; pool reflects contemporaneous known standings.

Worked Example: Mixed fwd/bwd in a Run

Goal: Use **individual forward TB ratio** but **senior rater pool backward mean** in the same Cox run.

1. In `pipeline_config.py` (or `pipeline_config_15_x.py`), include the columns:
   - Individual: `tb_ratio_fwd_snr`
   - Pool: `pool_minus_mean_snr_bwd`
2. In `COLUMN_CONFIG` (time varying list), include both:

```
tb_ratio_fwd_snr
pool_minus_mean_snr_bwd
```

3. In plot specs, point `variable` to the desired one for CR/CIF:

```
variable: tb_ratio_fwd_snr
```

or

```
variable: pool_minus_mean_snr_bwd
```

Algorithm 1: OER Assignment (Cell 4)

Goal: Attach each snapshot to the correct OER in both directions.

- Forward: most recent OER with `eval_thru_dt <= snpsht_dt`
- Backward: next OER with `eval_thru_dt >= snpsht_dt` (painted backward across rating period)

Core Steps: 1. Clean and normalize OER dates (`eval_strt_dt`, `eval_thru_dt`). 2. Deduplicate OER rows by officer and eval window. 3. For each officer: - Sort OERs by `eval_thru_dt`. - Use binary search to find last OER before each snapshot (forward). - Use binary search to find next OER after each snapshot (backward). 4. Compute cumulative counts and TB counts for rater and senior rater, separately for fwd and bwd. 5. Compute ratios as `cum_tb / cum_evals` using mode specific counts.

Important rule: If a rater or senior rater did not check a box on an OER, that OER does not increment `cum_evals` for that rater type.

Algorithm 2: Pool Means/Minus Means/Sizes (Cell 5)

Goal: Compare each officer to their rater/senior rater pool using the mode specific ratio selected for pool computation.

Core Steps: 1. Define pool keys: `(snpsht_dt, rtr_rater)` and `(snpsht_dt, snr_rater)`. 2. Group once per pool key and compute: - Pool mean TB ratio (fwd and bwd, rtr and snr) - Pool size 3. Compute minus mean as `individual_ratio - pool_mean`.

Single Pass Pattern: Pool stats are computed using one groupby per pool key and reused for mean, size, and minus mean to minimize repeated computation.

Algorithm 3: Pool Ranks/Percentiles/Z Scores (Cell 6)

Goal: Provide within pool relative standing.

Core Steps: 1. Group by pool keys, using the same pool definitions as Cell 5. 2. Compute: - Rank (ascending so higher percentile means better rank) - Percentile rank - Pool z score (within pool standardization) 3. Store results as permanent columns for downstream modeling.

Algorithm 4: Survival Interval Creation (Cell 10)

Goal: Convert snapshot records into time varying survival intervals.

Core Steps: 1. For each officer, order snapshots by date. 2. Create intervals from one snapshot to the next: - `start_time`: days since `dor_cpt` to interval start - `stop_time`: days since `dor_cpt` to interval end 3. Assign events: - Promotion: event=1 at `dor_maj` - Attrition: event=2 if officer leaves before study end - Censored: event=0 otherwise 4. Attach time varying covariates from the snapshot that defines the interval start.

Algorithm 5: Standardization and Interaction Terms (Cell 10.5)

Goal: Create standardized features and interaction terms for modeling.

Core Steps: 1. If standardization is enabled: - Compute z scores for configured columns. 2. If interaction terms are configured: - Build products from specified left/right columns. 3. Persist a clean modeling dataset for Cell 11/12.

Key Algorithmic Patterns

1. Temporal lookup pattern (Cell 4):
    - Sort OERs per officer by `eval_thru_dt`.
    - Use `np.searchsorted` for O(log N) lookup of last or next OER.
2. Interval overlap join pattern (Cell 4):
    - Join on officer ID to get candidate pairs.
    - Filter pairs with `eval_strt_dt <= snpsht_dt <= eval_thru_dt`.
    - Select most recent OER if multiple overlap.
3. Single pass pool grouping (Cells 5-6):

- Compute pool statistics in one groupby and reuse for derived columns.
- Reduces repeated groupby overhead across large snapshot sets.

4. Vectorized cumulative aggregation (Cell 4):
    - Precompute cumulative counts per officer.
    - Map to snapshot dates using binary search indices.

Phase Details (Cells 1 to 12.7)

Phase 1: Data Loading and Base Prep

Cell 1: Load raw snapshots - Inputs: 502 output or database/file - Output: `df_pipeline_01_raw`

Cell 2: Base filtering - Filters to commissioned officers and removes problematic job codes. - Output: `df_pipeline_02_base`

Cell 3: Year group (yg) - Derives yg from appointment dates, excluding inconsistent records. - Output: `df_pipeline_03_base`

Phase 2: OER Assignment and Pool Metrics

Cell 4: Assign OERs (fwd/bwd) - Inputs: `df_pipeline_03_base`, `df_oer_enriched` - Output: `df_pipeline_04a_basic_metrics` - Adds `cum_evals_*`, `cum_tb_*`, `tb_ratio_*` for rtr and snr in fwd and bwd modes.

Cell 5: Pool means/minus means/sizes - Inputs: `df_pipeline_04a_basic_metrics` - Output: `df_pipeline_05_pool_means` - Computes pool mean, minus mean, and pool size for rtr and snr pools.

Cell 6: Pool ranks/pct/z scores - Inputs: `df_pipeline_05_pool_means` - Output: `df_pipeline_06_pool_ranks` - Computes rank, percentile, and pool z score.

Phase 3: Variable Creation

Cell 7: Time varying variables - Adds prestige/unit/division/job code fields, and other time varying covariates. - Output: `df_pipeline_07_time_varying`

Cell 8: Static variables - Adds officer level covariates (sex, age at rank, final job code, etc.). - Output: `df_pipeline_08_combined`

Phase 4: Filtering

Cell 9: Advanced filtering - Controlled by `filtering_params`. - Output: `df_pipeline_09_filtered`

Phase 5: Cox Analysis

Cell 10: Survival interval creation - Output: `df_pipeline_10_cox_ready`

Cell 10.5: Standardize + interactions - Output: `df_pipeline_10_5_cox_zscored`

Cell 11: Plots and analysis dataset - Output: `df_pipeline_11_cox_analysis` - Builds `df_analysis` from basic_demographic_cols, static_cols, time_varying_cols, and **model_time_varying_cols** (so interaction/quadratic columns from the run config are available for CR plots).

Cell 12: Cox regression - 12.1 prepares data and validates structure. - 12.2 fits static model. - 12.3 fits full model (static + time varying). - 12.4 compares models and signal ratios. - 12.5 competing risks analysis. - 12.6 partial effects plots. - 12.6A combined/diagnostic effect plots. - 12.7 interaction diagnostics (placeholder structure when applicable).

Plotting System (Cell 11)

Competing Risks and CIF Bar Plots

- CIF bar charts are generated by `plot_competing_risks_cif_bars()` in `cox_plot_helpers.py`.
- Bar colors can be defined in plot spec using hex palettes:
  - `cif_bar_palette` (shared)
  - `cif_bar_palette_promo` / `cif_bar_palette_attr` (separate)
- Line plots use the same palette mapping so colors match between line and bar plots.

Plot Flags

- `plot_cif_bars`: enable CIF bars on a CR plot spec
- `cif_bar_show_legend`: toggle legend for CIF bars
- `cif_bar_legend_outside`: legend placement
- `include_z_cols`: auto include z cols for plotting when enabled
- `log_x`, `log_y`: optional log scaling in `plot_var_distribution()`

Quadratic Terms and CIF Bar Alignment

When the CIF bar charts show a U shape or inverted U shape not captured by a linear term, add a **quadratic term** to the Cox model:

1. Use the **raw** base variable (not z scored) to create the squared term.
2. Standardize the squared term if standardization is enabled.
3. Include both linear and squared terms in the model to allow curvature.

This is handled in Cell 10.5 via the interaction/standardization pipeline and validated in Cell 12.6A with combined effect plots (linear + quadratic) to compare the model curve against the CIF bar shape. **Cell 12.6A** detects the linear+quadratic pair from `final_model_vars` (or optional `COMBINED_EFFECT_BASE_COL` in config) so it works for any run (raw or z, pool minus mean or pool rank).

Performance Characteristics (Typical)

Phase	Cell	Typical Runtime	Time Complexity	Bottleneck
Phase 1	1-3	< 5 minutes	O(N)	I/O
Phase 2	4	20-40 minutes	O(M log M + S log M_o)	Temporal lookups
Phase 2	5	20-60 minutes	O(S log P)	Groupby aggregation
Phase 2	6	20-60 minutes	O(S log P)	Ranking and z scores
Phase 3	7-8	5-15 minutes	O(S)	Column creation
Phase 4	9	< 5 minutes	O(S)	Filtering
Phase 5	10	10-30 minutes	O(O x I)	Interval creation
Phase 5	10.5	5-15 minutes	O(S)	Standardization
Phase 5	11	5-20 minutes	O(G x T)	Plotting
Phase 5	12	10-60 minutes	O(N x V^2)	Cox fitting

Legend (used in the tables above): - N = total rows - S = snapshot rows - M = OER rows - M_o = avg OERs per officer - O = unique officers - I = avg intervals per officer - P = pool size - G = plot groups - T = time points - V = variables

Memory Characteristics (Typical)

Phase	Peak Memory	Notes
Phase 1	~2-5 GB	Raw snapshots
Phase 2	~10-20 GB	OER merge + fwd/bwd columns
Phase 2	~15-25 GB	Pool groupby operations
Phase 3-4	~10-15 GB	Variable creation + filters
Phase 5	~5-10 GB	Cox ready intervals

Memory optimization strategies: - Progressive saves allow early freeing of large DataFrames. - Column pruning before heavy groupby steps. - Run specific overrides to reduce unnecessary columns.

Inputs and Outputs (Quick Reference)

Primary Inputs

- Snapshots: `df_pipeline_01_raw` (or 502 output)
- OERs: `df_oer_enriched.feather` (from 512 pipeline)

Key Outputs

- `df_pipeline_04a_basic_metrics.feather`
- `df_pipeline_05_pool_means.feather`
- `df_pipeline_06_pool_ranks.feather`
- `df_pipeline_10_cox_ready.feather`
- `df_pipeline_10_5_cox_zscored.feather`
- `df_pipeline_11_cox_analysis.feather`
- `df_pipeline_12_01_prepared.feather`

512 Integration (OER Enrichment)

`df_oer_enriched.feather` is produced by `512_oer_int_working.ipynb` and includes rater and senior rater UICs needed for pool definitions. The 520 pipeline expects this file to exist and will load it directly in Cell 4.

Important Files

- `add_cum_oer_metrics_mod_working.py`: OER assignment + pool metrics (fwd/bwd only)
- `cox_plot_helpers.py`: Plotting utilities (CR, CIF bars, palettes, metadata cards)
- `pipeline_config.py`: Base configuration
- `pipeline_config_15_x.py`: Run specific overrides

Dependencies

- `numpy`, `pandas`
- `scikit-survival` (`sksurv`) for Cox regression and competing risks
- `matplotlib`, `seaborn` for plotting
- Optional: `modin`, `ray` for parallelism
- Optional: `tqdm` for progress bars

Usage Workflow and Troubleshooting

First Time Run

1. Set all `CELL*` flags to True in Cell 0.
2. Run cells sequentially (Cell 0 through Cell 12).

3. Review plots in output directories and model outputs in `cox_models` / `cox_results`.

Iterative Development

1. Set flags for cells you want to re run to True.
2. Set flags for cells you want to skip to False.
3. Rerun only necessary cells and reuse intermediate files.

Troubleshooting

1. If a cell fails, check the prior checkpoint output file.
2. Verify required columns exist before proceeding.
3. Use verbose logging and timing helpers to identify bottlenecks.

Notes and Principles

- Pool metrics are computed only in Cells 5-6.
- All metrics are forward/backward explicit; no legacy or base variants.
- Overrides are the right place to tune model variables and plots for a given run.